

IST602 Lecture 3 — XML (2025–2026) Detailed Summary

1) Big-picture purpose of XML

XML (eXtensible Markup Language) is presented as an open, non-proprietary standard for **describing and exchanging structured data** across systems, especially over the Internet.

Core ideas:

- XML separates **data meaning/structure** from display.
- It is both **human-readable** and **machine-processable**.
- It supports domain-specific vocabularies (e.g., XHTML, MathML, VoiceXML, CML, XBRL), enabling interoperability among heterogeneous applications and industries.

In practical terms, XML is a neutral transport/representation layer that allows independent applications to communicate with shared structure and semantics.

2) XML document fundamentals

2.1 Element and tag model

- Data is represented as **elements** enclosed by start/end tags.
- Each document must have exactly **one root element** containing all other elements.
- Empty elements can be represented as:
 - `<tag></tag>` or
 - self-closing `<tag />`.

2.2 Syntax rules and well-formedness

The lecture emphasizes that XML parsers accept only **well-formed** documents. Key rules:

- Exactly one root element.
- Properly nested tags.
- Every non-empty element has closing tag.
- Case-sensitive tags.
- Attribute values must be quoted.

2.3 Prolog, declaration, comments, whitespace

- Optional XML declaration may appear at top (before other markup).
 - Comments use `<!-- ... -->`.
 - Whitespace/indentation improve readability (typically parser-ignored except where content-sensitive).
-

3) XML processing ecosystem

3.1 XML parsers

- Parser validates syntax against XML rules.
- Exposes document data/structure to applications.
- A non-well-formed document is rejected for structured access.

3.2 Transformation and querying tools

- **XSLT**: transforms XML into other forms (often HTML for browser display).
- **XPath**: selects nodes/paths in XML trees.
- **XQuery**: query language for XML (analogous role to SQL for relational data).

4) Tree structure and navigation mindset

XML documents are hierarchical trees:

- Parent/child relationships.
- Sibling relationships.
- One parent per node, many children possible.

This mental model is foundational for XPath/XQuery and schema constraints.

5) Namespaces and name-collision control

Problem addressed: different vocabularies can use same tag names (e.g., `table`) with different semantics.

Solution:

- Use namespace URIs declared via `xmlns`.
- Use prefixes (`prefix:element`) to disambiguate.
- Optionally define a default namespace to reduce prefix repetition.

Namespaces let multiple vocabularies coexist safely in one document.

6) Well-formed vs valid XML

Important distinction:

- **Well-formed** = syntactically correct XML.
- **Valid** = well-formed **and** conforms to a declared grammar/contract (DTD or XSD).

Validation checks:

- Required elements and order.
- Allowed attributes.
- Data shape/types (stronger with XSD).

7) DTD (Document Type Definition)

DTD is introduced as the original XML grammar mechanism.

7.1 Internal vs external DTD

- Internal: `<!DOCTYPE ... [ ... ]>` inside XML file.
- External: separate `.dtd` file referenced from XML.

7.2 DTD building blocks

- Elements
- Attributes (`ATTLIST`)
- Entities
- `#PCDATA`
- `CDATA`

7.3 Content models and occurrence controls

DTD supports structure constraints such as:

- Exact sequence (`a,b,c`)
- Choice (`a|b`)
- Repetition:
 - `+` one or more
 - `*` zero or more
 - `?` zero or one
- `EMPTY`, `ANY`, mixed content.

7.4 Attribute constraints

- Types (e.g., `CDATA`, `ID`, `ENTITY`)
- Presence/value controls:
 - `#REQUIRED`
 - `#IMPLIED`
 - `#FIXED`

8) PCDATA vs CDATA (critical exam concept)

- **#PCDATA**: parsed character data; parser processes entities/markup.
 - Special chars (`<`, `>`, `&`) must be escaped (`<`, `>`, `&`).
- **CDATA**: parser does not interpret markup inside as tags/entities.

This distinction prevents accidental markup interpretation and controls parser behavior.

9) XML Schema (XSD) as modern validation model

XSD is presented as an XML-based successor/alternative to DTD with richer typing.

9.1 Schema mechanics

- `xs:schema` root in XML Schema namespace.
- `targetNamespace` defines vocabulary namespace governed by schema.
- Instance document references schema for validation.

9.2 Simple vs complex types

- **Simple types:** text-like values (`string`, `integer`, `date`, etc.), no child elements.
- **Complex types:** can include child elements and/or attributes.

9.3 Constraints and facets

- Value restrictions (ranges, enumerations, min/max, etc.).
- Default/fixed values.
- Required/optional attributes.
- Sequence/order and occurrence constraints (`minOccurs`, `maxOccurs`).

9.4 Advanced complex-content modeling

- Complex types with simple content.
- Complex types with complex content.
- Extensions/restrictions for reusable schema design.

10) Lecture's practical examples and what they teach

10.1 Letter + DTD example

Demonstrates:

- Root-level document contract.
- Ordered child elements.
- Use of attributes and empty elements.
- Why a structurally required element can appear even when semantically "empty".

10.2 Book / laptop + XSD examples

Demonstrate:

- User-defined types.
- Namespace linking between schema and instance.
- Complex type composition and reusable definitions.
- Element cardinality and semantic constraints.

11) Strengths and limits emphasized

Strengths

- Platform-neutral structured data exchange.
- Strong validation capability (especially XSD).
- Extensible vocabularies for many domains.

- Mature parser/tooling ecosystem.

Limits / trade-offs

- More verbose than lightweight formats.
 - Requires grammar discipline (DTD/XSD) for robust interoperability.
 - Display is not native XML's job; needs transformation/rendering logic.
-

12) Study checklist (high-yield)

Be able to:

1. Explain root element, proper nesting, and well-formedness rules.
 2. Distinguish well-formed vs valid XML.
 3. Use and justify namespaces.
 4. Explain DTD content models and occurrence operators (+, *, ?).
 5. Differentiate #PCDATA vs CDATA.
 6. Describe XSD simple/complex types and restrictions.
 7. Explain role differences: XSLT vs XPath vs XQuery.
 8. Interpret a schema-bound XML example and identify validation failures.
-

13) One-line takeaway

This lecture positions XML as a rigorous, schema-driven way to model and exchange structured information, with DTD/XSD enforcing contract integrity and tools like XPath/XQuery/XSLT enabling access, querying, and transformation.